Introduction to Algorithms

Sheet 2

Time Complexity

What is the time complexity of the following algorithms?

Algorithm 1

Finally -> O(n^2)

```
int t = 0;
for (int i = 1; i <= n; i++)                n
    for (int j = 0; j * j < 4 * n; j++)     j^2 = 4*n  -> j = 2sqrt(n)
        for (int k = 1; k * k <= 9 * n; k++)
            t++;                            k^2 = 9n -> k = 3sqrt(n)
```

Algorithm 2

```
int x = 0;
for (int i = 4 * n; i >= 1; i--)    O(n)
  x = x + 2 * i;
```

Algorithm 3

```
int z = 0;    Finall = O(log(n))
int x = 0;
for (int i = 1; i <= n; i = i * 3) {    1 3 9 27 ... n
  z = z + 5;
  z++;                                  i^k = n
  x = 2 * x;
}                                       k = log(n)
```

Algorithm 4

```
int y = 0;
for (int j = 1; j * j <= n; j++)   j^2 = n -> O(sqrt(n))
  y++;
```

Algorithm 5

i = 1 j run 1
i = 2 j run 2
i = 3 j run 3

```
int b = 0;   Finally O(n^2)
for (int i = n; i > 0; i--)
dependent Loop  for (int j = 0; j < i; j++)   so 1 + 2 + 3 + .... + n = n(n+1)/2
    b = b + 5;                                O(n^2)
```

Algorithm 6

```
int y = 1;   Finally -> O(n) + O(n) = O(n)
int j = 0;
for (j = 1; j <= 2n; j = j + 2)   from here j end at 2n   O(n)
  y = y + i;
int s = 0;
dependent loop for (i = 1; i <= j; i++)   so i <= 2n  O(n)
  s++;
```

Algorithm 7

from -> n + 2n + 3n + ... + n^n = n(1 + 2 + 3 .. + n)
= n * (n(n+1)/2) = n * (n^2 + n) = n^3

```
int b = 0;   Finall -> O(n^3)
for (int i = 0; i < n; i++)   will take n
dependent loop for (int j = 0; j < i * n; j++)
    b = b + 5;   for each i will take  sum(i*n)  at i = n it take n^2
```

Algorithm 8

```
int x = 0;   Finally -= O(n)
for (int i = 1; i <= n; i = i * 3) {
  if (i % 2 != 0)   always run because i is always odd
dependent loop for (int j = 0; j < i; j++)
      x++;   will run times sum(i)  so it will be O(n)
}   1 + 3 + 9 + ... + 3^k = Geometric Series
```

Algorithm 9

Finally -> O(n^4)

```
for (int i = 0; i <= n; i++)
    for (int j = 1; j <= i * i; j++)
      if (j % i == 0)
        for (int k = 0; k < j; k++)
          sum++;
```

n

n^2

for i^2 is divisble only when i not all i^2 so it iterate n times

n * n^2 * n = O(n^4)

Algorithm 10

```
int a = 0;
int k = n * n;
while (k > 1) {
  for (int j = 0; j < n * n; j++) {
    a++;
  }
  k = k / 2;
}
```

log(n)

o(n^2)

O(n^2 log(n))

Algorithm 11

```
int fun(int n) {
  int count = 0;
  for (int i = n; i > 0; i /= 2)
    for (int j = 0; j < i; j++)
      count += 1;
  return count;
}
```

Finally -> O(n)

dependent loop

1 -> n/2
2 -> n/4
3 -> n/8

n+n/2+n/4+n/8 1

n*[1+1/2+1/4+....]
approximate = 2 * n

Algorithm 12

```
int fun(int n) {
  int count = 0;
  for (int i = 0; i < n; i++)
    for (int j = i; j > 0; j--)
      count = count + 1;
  return count;
}
```

O(n^2)

1 -> 0
2 -> 1
3 -> 2

n -> n-1

0 + 1 + 2 + ... + n-1

Algorithm 13

```
int n, rev;  Finally = O(log(n))
rev = 0;
while (n > 0) {  ex: n = 1000      -> #1 = 1000
  rev = rev * 10 + n % 10;            #2 = 100
  n = n / 10;                         #3 = 10
}                                     #4 = 1
                  log1000 + 1= O(log(n))
```

Algorithm 14

```
int unknown(int n) {
  int i, j, k = 0;
  for (i = n / 2; i <= n; i++) n
    for (j = 2; j <= n; j = j * 2)  log(n)
      k = k + n / 2;
  return k;          Finally -> O(n log(n))
}
```

Algorithm 15

```
for (i = n, j = 0; i >0; i /= 2, j += i);  O(log(n))
```

Algorithm 16

```
int fun1(int n) {  Finally => O(n log(n))
  int i, j, k, p, q = 0;
  for (i = 1; i < n; ++i) {n
    p = 0;
    for (j = n; j > 1; j = j / 2)  log(n)
      ++p;  log(n)
    for (k = 1; k < p; k = k * 2)   log(p) = log(log(n))
      ++q;
  }
  return q;    finally -> n * [log(n) + log(log(n))] == n*log(n)
}
```

Algorithm 17

```
void fun() {    Finally -> O(n log(n))
  int i, j;
  for (i = 1; i <= n; i++)  n
    for (j = 1; j <= log(i); j++)  j = log(n)
      printf("GeeksforGeeks");
}
```

Algorithm 18

Finally = O(n) + O(n^2) = O(n^2)

```
int i = 0, j = 0, y = 0, s = 0;
for (j = 0; j < n + 1; j++)  n
  y = y + j;  0 + 1 + 2 + 3 + 4 + 5 ... n = n^2
for (i = 1; i <= y; i++)  n^2
  s++;
```

Algorithm 19

Finally = O(n)

```
int i = 1, z = 0;
while (z < n * (n + 1) / 2) {
  z += i;  1 + 2 + 3 + 4 ... k  ~=  k^2
  i++;
}          finally k^2 = n^2  so k = n
```